Zephyr RTOS: una aplicación, muchas placas

Devicetree, drivers estandarizados y portabilidad real

Fabrizio Carlassara
Responsable del Laboratorio de Sistemas Embebidos

Congreso de Ingeniería Electrónica 2026

UTN – Facultad Regional Avellaneda

Hoja de ruta

1. El problema: código atado al fabricante
2. Zephyr en cinco minutos
3. **Devicetree**: describir el hardware en vez de programarlo
4. **Modelo de drivers**: una API, muchas implementaciones
5. Demo: la misma aplicación en placas de cinco fabricantes
6. Herramientas, límites y cómo empezar

El problema

Prender un LED, cuatro fabricantes

STM32 (STM32Cube HAL)

```
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13,
                  GPIO_PIN_SET);
```

NXP (MCUXpresso SDK)

```
GPIO_PinWrite(GPIO, 1, 4, 1U);
```

Espressif (ESP-IDF)

```
gpio_set_level(GPIO_NUM_2, 1);
```

Raspberry Pi (Pico SDK)

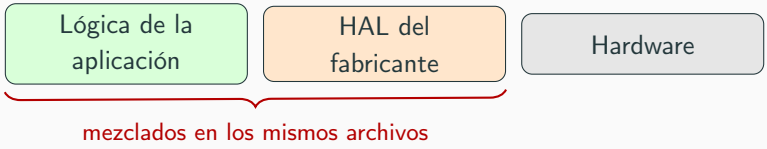
```
gpio_put(25, true);
```

Cuatro APIs, cuatro sistemas de build, cuatro formas de configurar relojes y pines.

El pin y el periférico quedan *escritos en el código*.

El costo de casarse con un fabricante

- **Faltantes de componentes** (2021–2023): rediseñar la placa *y* reescribir el firmware
- **Cambio de requerimientos**: más RAM, BLE, menor consumo… $\Rightarrow$ otro micro
- **Equipos y formación**: cada familia es un mundo nuevo
- **Prototipo vs. producto**: se prueba en una placa de desarrollo y se fabrica con otra



¿Y si la aplicación
no supiera en qué placa corre?

Zephyr en cinco minutos

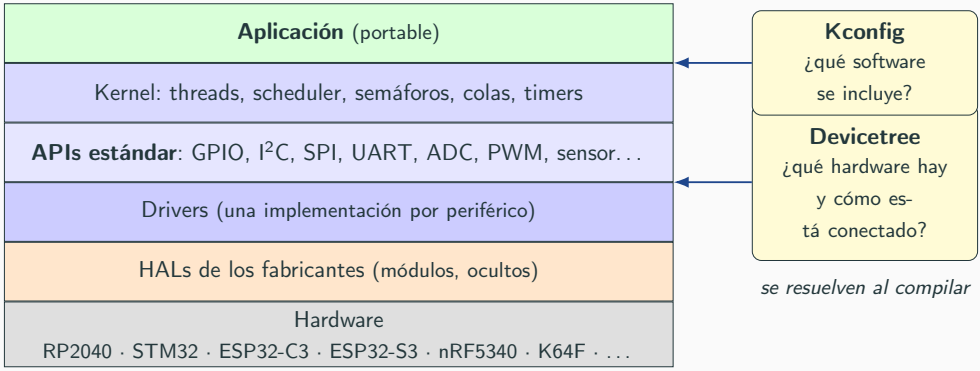
¿Qué es Zephyr?

- RTOS de código abierto, proyecto de la **Linux Foundation** (2016)
- Licencia **Apache 2.0**: sin regalías, apto para productos comerciales
- Gobernanza abierta: Intel, Nordic, NXP, ST, Espressif, Google, Meta...
- No es solo un kernel: **drivers, pilas de comunicación, sistema de build y herramientas**
- Versiones cada 6 meses, LTS con soporte extendido

Zephyr v4.4.2	
+1000	placas
160	fabricantes de placas
11	arquitecturas
+200	drivers de sensores

Contado sobre el árbol de v4.4.2

Arquitectura: dónde vive cada cosa



Zephyr vs. un RTOS "solo kernel"

	Kernel solo (p. ej. FreeRTOS)	**Zephyr**
Scheduler, colas, semáforos	Sí	Sí
Drivers de periféricos	Los del fabricante (HAL propio)	Propios, con API común
Descripción del hardware	En el código (`#define`, init)	Devicetree
Configuración	`FreeRTOSConfig.h`	Kconfig (`prj.conf`)
Build y flasheo	El del IDE del fabricante	`west` + CMake, igual para todos
Red, BLE, USB, FS	Bibliotecas externas	Integradas

No es mejor ni peor: **es otro alcance**. Zephyr ocupa más memoria y exige más formación.

Devicetree

Devicetree: describir, no programar

- Estructura de datos **jerárquica** que describe el hardware: buses, periféricos, pines, direcciones
- Viene de Open Firmware y del kernel Linux (ARM, 2011)
- En Linux se interpreta *al arrancar*; en Zephyr se resuelve al compilar
  - No hay parser en el microcontrolador
  - Todo termina en **macros de C** $\Rightarrow$ costo cero en tiempo de ejecución
  - Los errores (pin inexistente, propiedad faltante) aparecen **al compilar**

Anatomía de un nodo

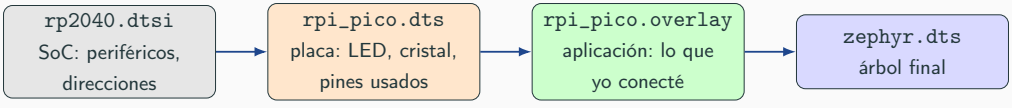
```
/ {
    aliases {
        sensor0 = &mpu6050;
    };
};

&i2c1 {
    status = "okay";
    clock-frequency = <400000>;

    mpu6050: mpu6050@68 {
        compatible = "invensense,mpu6050";
        reg = <0x68>;
    };
};
```

`/`	nodo raíz
`&i2c1`	referencia a un nodo ya definido (por su *label*)
`mpu6050:`	*label*: nombre para referenciarlo
`@68`	*unit address*: dirección en el bus
`compatible`	qué es el dispositivo ⇒ qué driver lo maneja
`reg`	dirección I^2C
`status`	habilitado o no
`aliases`	nombres genéricos que usa la aplicación

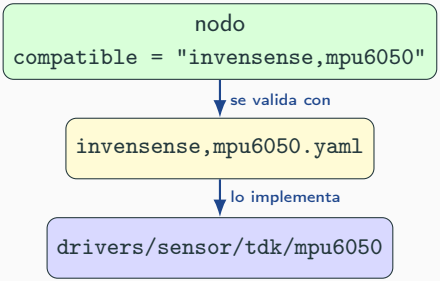
Capas: del silicio a la aplicación



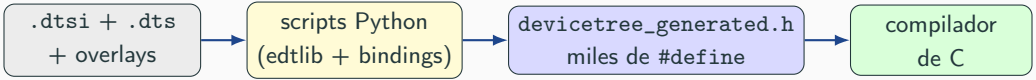
Cada capa **agrega o modifica** nodos de la anterior. La aplicación nunca toca las dos primeras.

Bindings: el contrato entre el árbol y el driver

- Archivo YAML por cada `compatible`
- Define qué propiedades existen, su tipo y si son obligatorias
- El build **valida** el árbol contra el binding
- Zephyr v4.4.2 trae más de 3700 bindings



Del árbol al código C




```c
/* La aplicación pide "lo que sea que esté en led0"... */
static const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);

/* ...y el preprocesador lo resuelve, por ejemplo en la Pico, a:
 *   { .port = &gpio0_device, .pin = 25, .dt_flags = GPIO_ACTIVE_HIGH }
 * Sin tablas en RAM, sin búsquedas en tiempo de ejecución. */
```

Kconfig: la otra mitad de la configuración

prj.conf de la app del acelerómetro

```
# APIs que usa la aplicación
CONFIG_GPIO=y
CONFIG_I2C=y
CONFIG_SENSOR=y

# Logging (con punto flotante)
CONFIG_LOG=y
CONFIG_CBPRINTF_FP_SUPPORT=y

# Shell para explorar el hardware
CONFIG_SHELL=y
CONFIG_I2C_SHELL=y
CONFIG_SENSOR_SHELL=y
```

- **Devicetree**: qué hardware hay
- **Kconfig**: qué software se compila
- Los drivers se activan solos: si hay un nodo `"invensense,mpu6050"` con `status = "okay"`, se compila su driver
- Un mismo `prj.conf` para todas las placas
- `west build -t menuconfig` para explorar

Modelo de drivers

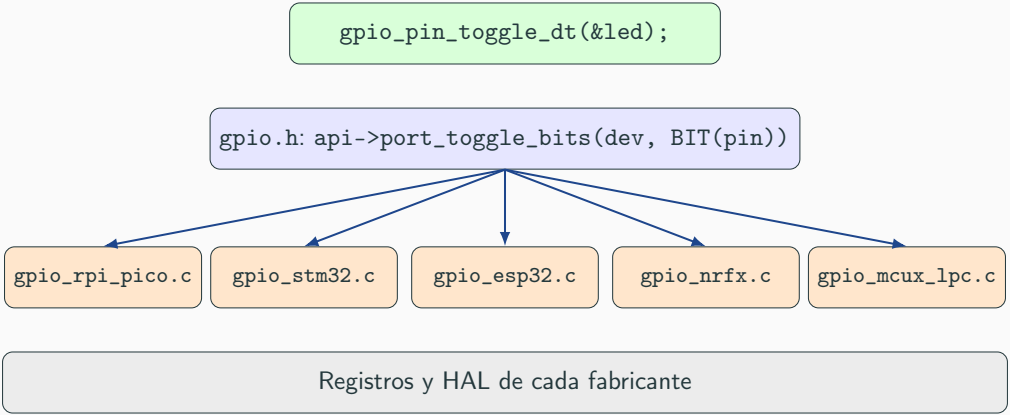
`struct device`: todo dispositivo es igual

```
struct device {
    const char *name;
    const void *config; /* flash */
    void *data;         /* RAM   */
    const void *api;    /* vtable */
    /* ... */
};
```

config: datos del devicetree (constantes)
data: estado del driver
api: tabla de funciones del driver

- **Una instancia por nodo** del devicetree con `status = "okay"`
- Se crean **al compilar** (`DEVICE_DT_INST_DEFINE`)
- Se inicializan solas antes de `main()`, por niveles: `PRE_KERNEL`, `POST_KERNEL`...
- La app obtiene el puntero con `DEVICE_DT_GET(nodo)`

Una API, muchas implementaciones



La función que se ejecuta la decide **el devicetree**, a través del puntero `api` del dispositivo.

La API de sensores

```
const struct device *s =
        DEVICE_DT_GET(DT_ALIAS(sensor0));
struct sensor_value v[3];

sensor_sample_fetch(s);  /* medir */

sensor_channel_get(s,
        SENSOR_CHAN_ACCEL_XYZ, v);  /* m/s2 */
sensor_channel_get(s,
        SENSOR_CHAN_GYRO_XYZ, v);   /* rad/s */
/* canal que no mide -> -ENOTSUP */
```

- Canales con **unidades del SI**: m/s^2, rad/s, gauss, °C
- `sensor_value`: entero + millonésimas, sin `float` obligatorio
- Igual para MPU6050, FXOS8700, MMA8652FC y otros +200
- API asíncrona `sensor_read()` + decodificadores para alto rendimiento

APIs estandarizadas: no solo GPIO

Periféricos	**Comunicación**	**Servicios**
GPIO, PWM, ADC, DAC	UART, I^2C, SPI, CAN	Logging, shell
Contadores, watchdog, RTC	USB device/host	Sistema de archivos
Sensores, LED, displays	Bluetooth LE	Settings (NVS)
Entrada (teclas, touch)	Ethernet, Wi-Fi, TCP/IP	Gestión de energía
Flash, EEPROM	Thread, Matter, LoRa	Actualización (MCUboot)

Mismo patrón en todas: `DEVICE_DT_GET` + `device_is_ready` + funciones de la API.

Demo

El banco de pruebas

Placa	Fabricante	Núcleo	Acelerómetro en la demo
Raspberry Pi Pico	Raspberry Pi	Cortex-M0+	MPU6050 externo
Black Pill F401	STMicroelectronics	Cortex-M4F	MPU6050 externo
ESP32-C3 SuperMini	Espressif	RISC-V	MPU6050 externo
ESP32-S3 DevKitC	Espressif	Xtensa LX7	MPU6050 externo
nRF5340 DK	Nordic	Cortex-M33	MPU6050 externo
FRDM-K64F	NXP	Cortex-M4F	FXOS8700 integrado *y* MPU6050

Cinco fabricantes, tres arquitecturas: ARM, Xtensa y RISC-V.

El workspace también compila para Pico 2, DOIT ESP32 DevKit V1, LPCXpresso55S69 y FRDM-MCXN236.

App 1: `blinky_boton` — el `main.c` (lo esencial)

```c
static const struct gpio_dt_spec led   = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
static const struct gpio_dt_spec boton = GPIO_DT_SPEC_GET(DT_ALIAS(sw0),  gpios);

int main(void)
{
    gpio_pin_configure_dt(&led, GPIO_OUTPUT_INACTIVE);
    gpio_pin_configure_dt(&boton, GPIO_INPUT);
    gpio_pin_interrupt_configure_dt(&boton, GPIO_INT_EDGE_TO_ACTIVE);
    gpio_init_callback(&boton_cb, boton_presionado, BIT(boton.pin));
    gpio_add_callback_dt(&boton, &boton_cb);

    while (1) {
        gpio_pin_toggle_dt(&led);
        k_msleep(periodos_ms[atomic_get(&indice)]);
    }
}
```

Ni un número de pin. Ni un registro. Ni un `#ifdef` de placa.

Lo que cambia por placa: el overlay

boards/rpi_pico.overlay (la Pico no tiene botón)

```
/ {
    botones {
        compatible = "gpio-keys";
        boton_ext: boton_ext {
            gpios = <&gpio0 15
              (GPIO_ACTIVE_LOW | GPIO_PULL_UP)>;
        };
    };
    aliases {
        sw0 = &boton_ext;
    };
};
```

nRF5340 DK, FRDM-K64F, ESP32-C3...

- Ya traen `led0` y `sw0` en su `.dts`
- No necesitan overlay

Compilar y grabar

```
west build -p -b rpi_pico apps/blinky_boton
west build -p -b esp32c3_supermini \
    apps/blinky_boton
west flash
```

App 2: acelerometro — no sabe qué sensor tiene

```
static const struct device *const sensor = DEVICE_DT_GET(DT_ALIAS(sensor0));

static const struct magnitud magnitudes[] = {
    {SENSOR_CHAN_ACCEL_XYZ, 3, "Aceleracion", "m/s2"},
    {SENSOR_CHAN_GYRO_XYZ,  3, "Giro",        "rad/s"},
    {SENSOR_CHAN_MAGN_XYZ,  3, "Campo magn.", "G"},
    {SENSOR_CHAN_DIE_TEMP,  1, "Temp. chip",  "C"},
};

while (1) {
    sensor_sample_fetch(sensor);
    for (size_t i = 0; i < ARRAY_SIZE(magnitudes); i++) {
        mostrar(&magnitudes[i]);  /* si el driver responde -ENOTSUP, se omite */
    }
    k_msleep(2000);
}
```

Placa + sensor: overlays que se combinan

boards/frdm_k64f.overlay (se aplica solo)

```
sensor_i2c: &i2c0 {       /* mi bus I2C */
    status = "okay";
};
/ { aliases {              /* integrado */
    sensor0 = &fxos8700; }; };
```

mpu6050.overlay (se pide al compilar)

```
&sensor_i2c {
    mpu6050: mpu6050@68 {
        compatible = "invensense,mpu6050";
        reg = <0x68>;
    };
};
/ { aliases { sensor0 = &mpu6050; }; };
```

```
# acelerómetro integrado
west build -p -b frdm_k64f apps/acelerometro

# MPU6050 externo, misma placa
west build -p -b frdm_k64f apps/acelerometro \
  -- -DEXTRA_DTC_OVERLAY_FILE=mpu6050.overlay

# MPU6050 externo, otra placa
west build -p -b blackpill_f401cc \
  apps/acelerometro \
  -- -DEXTRA_DTC_OVERLAY_FILE=mpu6050.overlay
```

El overlay extra se aplica **después** del de la placa: `sensor0` pasa a ser el MPU6050.

N placas + M sensores = $N + M$ archivos, no $N \times M$.

Mismo `main.c`, distinto sensor

Sensor	Dónde	Acel.	Giro	Magn.	Temp.
MPU6050	externo (cualquier placa)	✓	✓	✗	✓
FXOS8700	integrado en FRDM-K64F	✓	✗	✓	✗
MMA8652FC	integrado en LPC55S69	✓	✗	✗	✗

- La app pide las cuatro magnitudes; cada driver **responde lo que mide**
- FXOS8700 y MMA8652FC: dos chips, un solo driver (`nxp,fxos8700`)
- El modo (solo acelerómetro o acelerómetro + magnetómetro) lo fija el **Kconfig de cada placa**

Demo en vivo

Mismo `main.c`, cinco fabricantes,
tres arquitecturas, dos sensores

Resultados: lo que cambia y lo que cuesta

Placa	Núcleo	Sensor	Flash	RAM	Overlay
Raspberry Pi Pico	Cortex-M0+	MPU6050	90,7 KiB	17,0 KiB	3 + 11
Black Pill F401	Cortex-M4F	MPU6050	91,3 KiB	18,0 KiB	3 + 11
ESP32-C3 SuperMini	RISC-V	MPU6050	148,0 KiB	112,3 KiB	3 + 11
ESP32-S3 DevKitC	Xtensa LX7	MPU6050	148,5 KiB	110,8 KiB	3 + 11
nRF5340 DK	Cortex-M33	MPU6050	92,4 KiB	17,6 KiB	3 + 11
FRDM-K64F	Cortex-M4F	FXOS8700 integrado	90,7 KiB	17,9 KiB	8
FRDM-K64F	Cortex-M4F	MPU6050	92,3 KiB	17,9 KiB	8 + 11

`main.c` de `acelerometro`: 61 líneas, idéntico en todas las placas.

Overlay = placa + `mpu6050.overlay`. Memoria según el enlazador: kernel, drivers, logging y shell.

Explorar el hardware desde la shell

```
uart:~$ device list
uart:~$ i2c scan i2c@40005400
     0  1  2  3  4  5  6  7  8  9  a  b  ...
60: -- -- -- -- -- -- -- -- 68 -- -- --
uart:~$ sensor get mpu6050@68
uart:~$ kernel thread list
```

- La shell está en el `prj.conf`: viene en la app, sin recompilar nada
- Comandos para GPIO, I^2C, sensores, threads, memoria
- Ideal para validar el cableado antes de escribir código

Herramientas y límites

`west`: una sola herramienta para todo

```
west init -l cie2026-zephyr    # workspace
west update                    # Zephyr + HALs
west build -b <placa> <app>
west flash                     # grabar
west debug                     # depurar (GDB)
west build -t menuconfig       # Kconfig
```

- `west flash` elige el programador según la placa: openocd, pyocd, `esptool`, `dfu-util`, `nrfutil`, `linkserver`, J-Link…
- El **manifest** (`west.yml`) fija versiones ⇒ builds reproducibles
- Funciona igual en Linux, macOS y Windows

Lo que Zephyr *no* resuelve

- **Periféricos exclusivos** de un fabricante (PIO del RP2040, radio de Nordic, ULP del ESP32): portables solo en parte
- **Soporte desparejo**: no todos los drivers implementan todo (DMA, bajo consumo, *async*)
- **Costo en memoria**: más que bare-metal o un kernel solo (esta app: ≈90 KiB de flash en Cortex-M); en micros chicos no entra
- **Curva de aprendizaje**: devicetree, Kconfig, CMake y west de entrada
- **Temporización y consumo** siguen dependiendo del silicio: portar no es lo mismo que validar

La portabilidad no es gratis: se paga una vez, en la arquitectura, y no en cada cambio de placa.

Para llevarse

1. El **devicetree** separa *qué hardware hay* de *qué hace la aplicación*
2. Las **APIs estandarizadas** hacen que el driver sea un detalle de implementación
3. Cambiar de placa o de sensor = **otro overlay**, no otro `main.c`
4. El código de la aplicación se vuelve **reutilizable, testeable y enseñable**

Probalo vos

- Workspace completo: manifest, apps y overlays
- Guía paso a paso para tu computadora (Linux, macOS o Windows)
- Nota técnica con los resultados

`https://github.com/utn-fra-lse/cie2026-zephyr`



Laboratorio de Sistemas Embebidos
UTN – Facultad Regional Avellaneda

¿Preguntas?